

מבוא ללמידה עמוקה - תרגיל 3

חלק א' - החלק המעשי

בתרגיל זה אנו בונים רשתות שמבאות את רגש הצופה (חיובי/שלילי) על סמך ביקורת הסרט שכתב. כל ביקורת מקוצצת ל-100 מילים, וכל מילה ממופה לווקטור בן 100 מימדים באמצעות שיכון GloVe קפוא (כדי למנוע overfitting). השווינו ארבע אסטרטגיות: RNN פשוט, GRU, Global Average Pooling, ושכבת Self-Attention מקומית.

משימה 1 - רשתות רקורנטיות (RNN ו-GRU)

1.1 תא ה-RNN (Elman)

בכל צעד זמן התא מקבל את ווקטור המילה x_t ואת המצב החבוי הקודם h_{t-1} , משרשר אותם, מעביר בשכבה לינארית בודדת ומפעיל $\tanh$ כדי לקבל את h_t . בתום סריקת 100 המילים, המצב החבוי האחרון עובר בשכבת סיווג שמפיקה 2 logits (אחד לכל מחלקת רגש).

```
class ExRNN(nn.Module):
    def __init__(self, input_size, output_size, hidden_size):
        super(ExRNN, self).__init__()
        self.hidden_size = hidden_size
        self.in2hidden = nn.Linear(input_size + hidden_size, hidden_size)
        self.hidden2out = nn.Linear(hidden_size, output_size)

    def forward(self, x, hidden_state):
        combined = torch.cat((x, hidden_state), dim=1)
        hidden = torch.tanh(self.in2hidden(combined))
        output = self.hidden2out(hidden)
        return output, hidden

    def init_hidden(self, bs):
        return torch.zeros(bs, self.hidden_size,
                           device=next(self.parameters()).device)
```

1.2 תא ה-GRU

ה-GRU מוסיף שני שערים נלמדים: שער update השולט עד כמה לשמר את המצב הישן מול המועמד החדש, ושער reset השולט עד כמה "לשכוח" את המצב הקודם בעת חישוב המועמד. השערים מאפשרים זרימת מידע וגרדיאנט לאורך רצפים ארוכים, ובכך מקלים על בעיית ה-vanishing gradient.

```
class ExGRU(nn.Module):
    def __init__(self, input_size, output_size, hidden_size):
```

```

super(ExGRU, self).__init__()
self.hidden_size = hidden_size
self.update_gate = nn.Linear(input_size + hidden_size, hidden_size)
self.reset_gate = nn.Linear(input_size + hidden_size, hidden_size)
self.candidate = nn.Linear(input_size + hidden_size, hidden_size)
self.hidden2out = nn.Linear(hidden_size, output_size)

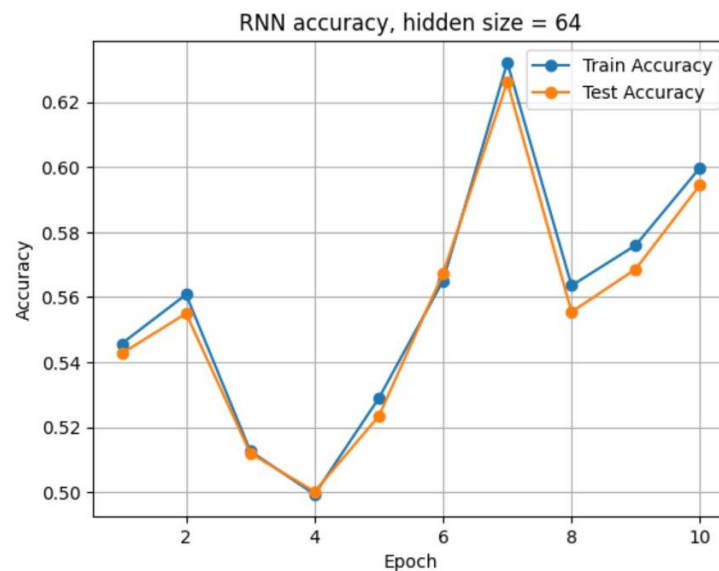
def forward(self, x, hidden_state):
    combined = torch.cat((x, hidden_state), dim=1)
    update = torch.sigmoid(self.update_gate(combined))
    reset = torch.sigmoid(self.reset_gate(combined))
    reset_hidden = reset * hidden_state
    candidate_combined = torch.cat((x, reset_hidden), dim=1)
    candidate_hidden = torch.tanh(self.candidate(candidate_combined))
    hidden = (1 - update) * hidden_state + update * candidate_hidden
    output = self.hidden2out(hidden)
    return output, hidden

```

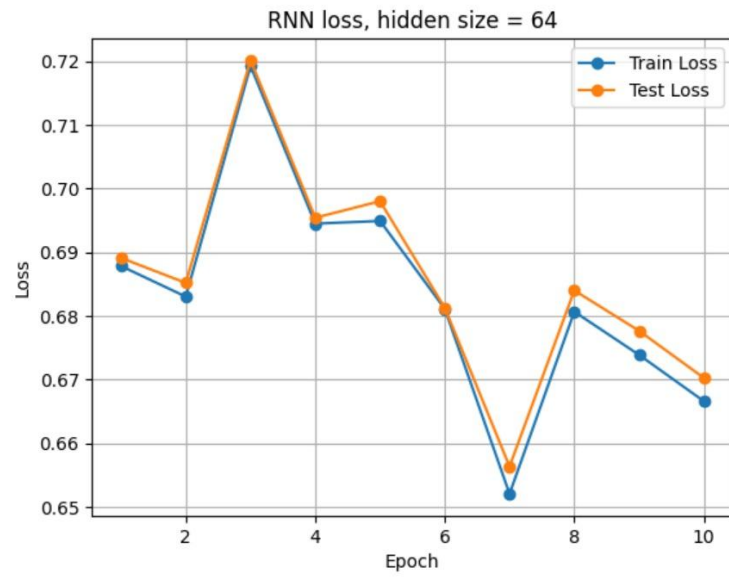
כפי שנתבקשנו, אנו מוציאים פלט של 2 logits (ולא סקלר יחיד) כדי שנוכל בהמשך (אסטרטגיות 3–4) למצע ציוני תת-ניבוי דו-מימדיים לכל מילה.

הרצנו כל אחת מהארכיטקטורות (RNN, GRU) עם שני גדלי מצב חבוי: **קטן (64) וגדול (128)**. כל הרצה נמשכה 10 epochs, עם optimizer מסוג Adam, learning rate 0.001, ופונקציית loss מסוג Cross-Entropy. להלן גרפי ה-accuracy וה-loss שהתקבלו, ולאחריהם סיכום וניתוח.

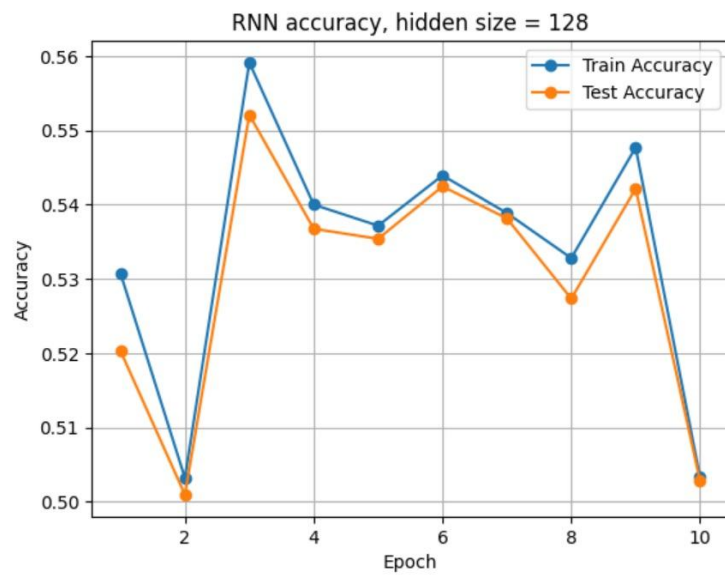
1.3 תוצאות RNN



accuracy - RNN, hidden size = 64



loss - RNN, hidden size = 64

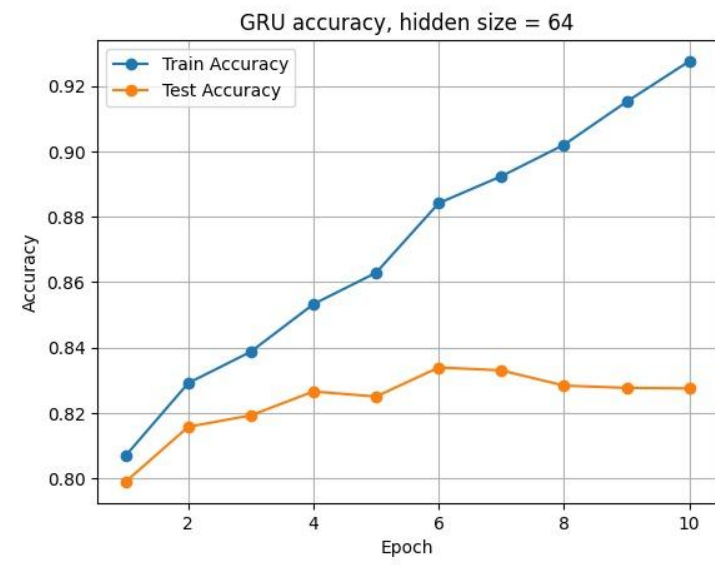


accuracy - RNN, hidden size = 128



loss - RNN, hidden size = 128

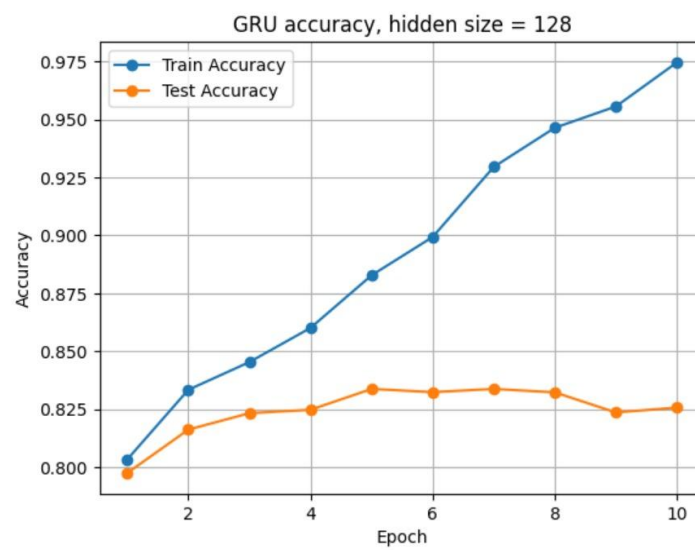
1.4 תוצאות GRU



accuracy - GRU, hidden size = 64



loss - GRU, hidden size = 64



accuracy - GRU, hidden size = 128



1.5 טבלת סיכום תוצאות

Test Loss	Train Loss	Test Acc	Train Acc	Hidden	מודל
0.413	0.193	0.828	0.927	64	GRU
0.6122	0.0770	0.8256	0.9746	128	GRU
0.670	0.667	0.595	0.600	64	RNN
0.7129	0.7110	0.5028	0.5033	128	RNN

1.6 ניתוח התוצאות

GRU לעומת RNN

ה-GRU מגיע ל-test accuracy של כ-0.83, בעוד ה-RNN הפשוט תקוע סביב 0.50–0.60 - קרוב לניחוש אקראי. הסיבה היא בעיית ה-vanishing gradient: ב-RNN המצב החבוי מוכפל שוב ושוב לאורך 100 צעדים, והגרדיאנט מתפוגג עד שהוא מגיע למילים המוקדמות, כך שהרשת אינה לומדת תלות ארוכת-טווח. ה-GRU פותר זאת בעזרת שערי update/reset, המאפשרים לשמר מידע לאורך הרצף ולהתעלם סלקטיבית ממידע לא-רלוונטי.

השפעת גודל המצב החבוי (64 לעומת 128)

ב-GRU שני הגדלים מגיעים ל-test accuracy דומה (0.83), אך עם hidden=128 ה-overfitting חזק יותר: ה-train loss צונח ל-0.08 בעוד ה-test loss עולה מ-0.37 (epoch 5) ל-0.61 (epoch 10). עם hidden=64 הפער קטן יותר וה-test loss מתייצב סביב 0.41–0.37. כלומר יותר פרמטרים = קיבולת גבוהה יותר אך גם נטייה חזקה יותר לשנן את מאגר האימון. ב-RNN שני הגדלים נכשלים כלומר הגדלת הקיבולת אינה עוזרת כשהגרדיאנט עצמו אינו זורם לאורך הרצף, כך שהבעיה מבנית ולא בעיית קיבולת.

1.7 ביקורת לדוגמה להדגמת ההבדל בין RNN ל-GRU

בנינו ביקורת מבחן הבדקת היפוך רגש לאורך הרצף (תלות ארוכת-טווח) - תכונה שבה צפוי ה-GRU להצליח וה-RNN להיכשל. הרעיון: לפתוח את הביקורת במשפט חיובי, ואז להפוך את הרגש בהמשך כך שהמסקנה הסופית שלילית. כדי לנבא נכון יש לזכור את ההיפוך ולשקלל נכון את סוף הרצף.

התוצאה המייצגת ביותר (תווית אמת: negative):

```
Review:
At first I liked the movie but after a while it became slow confusing and disappointing
RNN: positive, positive=0.5309, negative=0.4691
GRU: negative, positive=0.0248, negative=0.9752
```

ניתוח: הביקורת נפתחת חיובי ("At first I liked the movie") ומתהפכת בסוף ("slow confusing and disappointing"), כך שהרגש הכולל שלילי. ה-RNN נשאר חסר-החלטה (positive=0.5309) ואף טועה ומנבא חיובי - תואם לכך שהוא בקושי לומד (test accuracy 0.50). ה-GRU מנבא negative בביטחון גבוה (0.9752): השערים מאפשרים לו לעדכן את המצב כך שסוף הרצף השלילי הופך לדומיננטי. הדוגמה ממחישה את יתרון ה-GRU בטיפול בהיפוך רגש ובתלות ארוכת-טווח.

1.8 RNN - שילוב הקלט עם המצב החבוי והפעלת האקטיבציה

```
combined = torch.cat((x, hidden_state), dim=1)
hidden = torch.tanh(self.in2hidden(combined))
```

1.9 GRU - חישוב שער ה-reset ושער ה-update

```
update = torch.sigmoid(self.update_gate(combined))
reset = torch.sigmoid(self.reset_gate(combined))
```

משימה 2 - Token-wise MLP ו-Global Average Pooling

באסטרטגיה זו כל מילה עוברת באופן עצמאי דרך MLP קטן המפיק ווקטור דו-מימדי של logits - "ציון תת-ניבוי" (sub-prediction score). לאחר מכן ממצעים את כל הווקטורים כדי לקבל ניבוי סופי, מה שמאפשר טיפול בקלט באורך משתנה.

2.1 ארכיטקטורת ה-FC

השתמשנו ב-MLP בן שלוש שכבות עם אקטיבציית ReLU בין השכבות: $input \rightarrow 128 \rightarrow 128 \rightarrow 2$. (ניסינו מספר אפשרויות עבור גודל השכבה ולא היה הבדלים כמעט, בחרנו באפשרות זו כי היא היתה טובה במעט משאר האפשרויות אחרות שניסינו). השכבה האחרונה מפיקה 2 logits לכל מילה. עם hidden=128 הושג test loss נמוך (0.38).

```
class ExMLP(nn.Module):
    def __init__(self, input_size, output_size, hidden_size):
        super(ExMLP, self).__init__()
        self.ReLU = torch.nn.ReLU()
        self.layer1 = MatMul(input_size, hidden_size)
        self.layer2 = MatMul(hidden_size, hidden_size)
        self.layer3 = MatMul(hidden_size, output_size)

    def forward(self, x):
        x = self.layer1(x)
        x = self.ReLU(x)
        x = self.layer2(x)
        x = self.ReLU(x)
```

```
x = self.layer3(x)
return x
```

2.2 היכן מפעילים את ה-softmax ושורת ה-Global Average Pooling

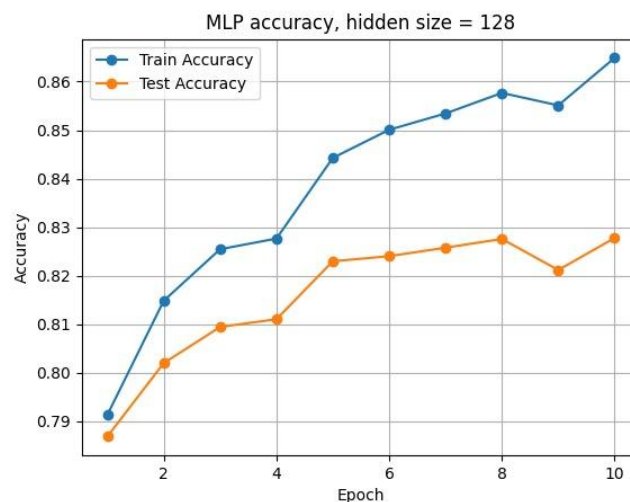
התשובה: אחרי המיצוע - ממצעים את ה-logits של כל המילים, ורק על הווקטור הממוצע מפעילים softmax.

נימוק: מיצוע ה-logits שומר על עוצמת האות של מילים אינפורמטיביות. אילו היינו מפעילים softmax לכל מילה בנפרד לפני המיצוע, היינו דוחסים כל ציון לתחום [0,1] ומאבדים את ההבדל בין מילה עם logit קיצוני (למשל "wonderful" עם +39) למילה חלשה - שתיהן היו תורמות כמעט באותה מידה. מיצוע ה-logits נותן למילים החזקות משקל גדול יותר, וגם תואם לכך ש-CrossEntropyLoss מצפה ל-logits.

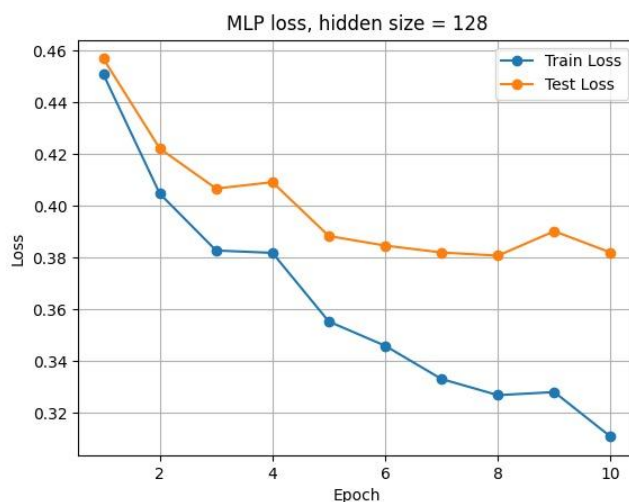
השורה שבה מבוצע ה-Global Average Pooling:

```
mask = (reviews.abs().sum(dim=-1) > 0).float()
expanded_mask = mask.unsqueeze(-1)
masked_sub_scores = sub_score * expanded_mask
output = masked_sub_scores.sum(dim=1) / mask.sum(dim=1).clamp(min=1).unsqueeze(-1)
```

2.3 תוצאות האימון (hidden size = 128)



accuracy - MLP, hidden size = 128



loss - MLP, hidden size = 128

הערכה מלאה לאחר האימון:

Full evaluation after training:

Train Loss: 0.3108, Train Acc: 0.8649

Test Loss: 0.3820, Test Acc: 0.8277

ה-MLP מגיע ל-test accuracy של 0.828 - דומה ל-GRU, ועם overfitting מתון בלבד (הפער בין train ל- test loss קטן). זאת למרות שהמודל מנקד כל מילה בנפרד וללא הקשר, מה שמלמד שעבור ניתוח רגשות חלק ניכר מהאות נושא ברמת המילה הבודדת.

2.4 ניתוח שגיאות - טבלאות sub-prediction score

לכל ביקורת מודפסת טבלת (מילה, ציון חיובי, ציון שלילי). הניבוי הסופי הוא הממוצע של הציונים על כל המילים.

ביקורת 1 - TP (true positive)

"This movie was wonderful and exciting with great acting and a beautiful story"

תויות אמת: positive · ניבוי: positive=1.0000, negative=0.0000

neg_score	pos_score	מילה
-0.1290	0.2064	this
0.4382	-0.4473	movie
1.4441	-1.3385	was
-42.2850	39.4231	wonderful
-3.7900	3.6693	and
-11.1929	10.0295	exciting
-0.7203	0.8733	with
-23.7080	22.5985	great
3.6496	-3.5750	acting
-3.7900	3.6693	and

neg_score	pos_score	מילה
-7.0589	6.7605	beautiful
-1.1441	1.1143	story

ביקורת 2 - TN (true negative)

"This movie was boring slow confusing and disappointing with terrible acting"

תווית אמת: negative · ניבוי: negative=1.0000, positive=0.0000

neg_score	pos_score	מילה
-0.1290	0.2064	this
0.4382	-0.4473	movie
1.4441	-1.3385	was
56.0766	-57.3529	boring
19.9113	-20.8055	slow
16.9740	-15.9973	confusing
-3.7900	3.6693	and
64.1545	-66.9471	disappointing
-0.7203	0.8733	with
53.6169	-55.7665	terrible
3.6496	-3.5750	acting

ביקורת 3 - FP (false positive)

"My friends said the movie was wonderful but my experience was the complete opposite"

תווית אמת: negative · ניבוי: positive=0.9998, negative=0.0002

neg_score	pos_score	מילה
-5.8035	4.9463	my
-9.2024	8.2292	friends
-0.3365	0.3514	said
-0.1086	0.1188	the
0.0286	-0.2844	movie
1.0442	-1.1113	was
-40.5629	36.0679	wonderful
-0.5542	0.6312	but
-5.8035	4.9463	my
-15.5956	13.8328	experience
1.0442	-1.1113	was
-0.1086	0.1188	the

neg_score	pos_score	מילה
9.1485	-9.8351	complete
0.7348	-1.1884	opposite

ביקורת 4 - FN (false negative)

"The beginning was slow and boring but later the story became moving beautiful and excellent"

תווית אמת: positive · ניבוי: negative=0.7131, negative=0.2869

neg_score	pos_score	מילה
0.1227	-0.0832	the
-2.0274	2.4903	beginning
1.4441	-1.3385	was
19.9113	-20.8055	slow
-3.7900	3.6693	and
56.0766	-57.3529	boring
-0.5551	1.0691	but
-2.4629	2.9926	later
0.1227	-0.0832	the
-1.1441	1.1143	story
-0.9281	1.0359	became
-0.4023	0.3254	moving
-7.0589	6.7605	beautiful
-3.7900	3.6693	and
-50.3075	48.0875	excellent

הסבר לתוצאות

ה-MLP מנקד כל מילה בנפרד, ולכן מילים בעלות מטען רגשי חזק שולטות בממוצע: "wonderful" (+39), "excellent" (+48) מצד אחד, ו-"terrible" (-56), "disappointing" (-67), "boring" (-57) מצד שני. מילות פונקציה ("the", "was", "and") מקבלות ציונים קרובים לאפס וכמעט אינן משפיעות.

ב-TP וב-TN המילים הדומיננטיות תואמות לתווית, ולכן הניבוי נכון ובוודאות גבוהה.

ב-FP (ביקורת 3) הביקורת שלילית בפועל ("my experience was the complete opposite" - ההפך מ-"wonderful"), אך המודל ניבא חיובי בוודאות גבוהה. הסיבה זהה: "experience" (+36), "wonderful" (+13.8) ו-"friends" (+8.2) נושאות ציון חיובי גבוה, בעוד המילים השוללות "complete" (-9.8) ו-"opposite" (-1.2) חלשות מכדי לאזן. ה-MLP אינו מבין שהצירוף "the complete opposite" הופך את משמעות "wonderful" - שוב, בהיעדר הקשר.

ב-FN (ביקורת 4) מתקבלת אותה מגבלה מהכיוון ההפוך: הביקורת חיובית בפועל ("became moving beautiful and excellent"), אך המילים השליליות בפתיחה - "boring" (-57) ו-"slow" (-21) - גדולות בערכן המוחלט וגוררות את הממוצע לשלילי, למרות "excellent" (+48). ה-MLP אינו רואה ש-"but later" מהפך את

משמעות הפתיחה. שתי השגיאות (FN ו-FP) נובעות מכך שה-MLP מנקד מילים בנפרד וללא סדר - בדיוק הבעיה ששכבת ה-Self-Attention (משימה 3) נועדה לפתור.

משימה 3 - שכבת Restricted Self-Attention

3.1 מימוש שכבת ה-Self-Attention

מימשנו את השכבה לפי שלד של `ExLRestSelfAtten`: כל מילה עוברת תחילה שכבת FC (layer1) ל-hidden, ואז נבנית סביבה מקומית של 11 מילים (5 מכל צד) באמצעות `torch.roll` וריפוד אפסים. מוסיפים positional encoding מקומי נלמד, ומחשבים attention עם ראש בודד (Q, K, V). המילה הנוכחית היא ה-Query, ושכנותיה הן ה-Keys/Values; הפלט הוא ממוצע משוקלל של ה-Values בחלון.

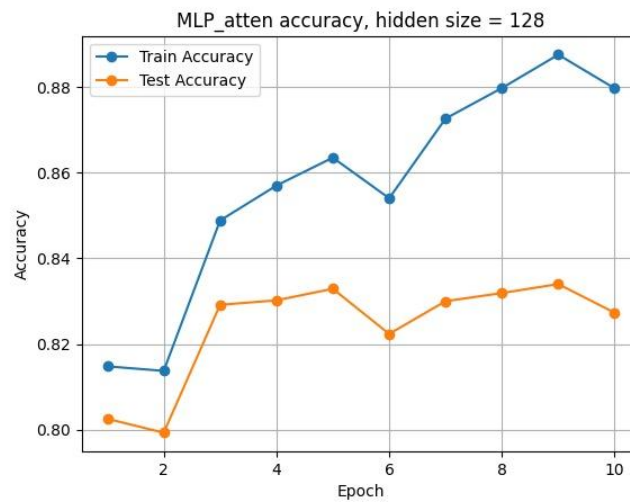
```
class ExLRestSelfAtten(nn.Module):
    def __init__(self, input_size, output_size, hidden_size):
        super().__init__()
        self.atten_size = atten_size
        self.window_size = 2 * atten_size + 1
        self.sqrt_hidden_size = np.sqrt(float(hidden_size))
        self.ReLU = torch.nn.ReLU()
        self.softmax = torch.nn.Softmax(dim=2)
        self.layer1 = MatMul(input_size, hidden_size)
        # learnable local positional encoding for offsets [-atten_size..+atten_size]
        self.pos_encoding = nn.Parameter(
            0.01 * torch.randn(1, 1, self.window_size, hidden_size))
        self.W_q = MatMul(hidden_size, hidden_size, use_bias=False)
        self.W_k = MatMul(hidden_size, hidden_size, use_bias=False)
        self.W_v = MatMul(hidden_size, hidden_size, use_bias=False)
        self.layer2 = MatMul(hidden_size, hidden_size)
        self.layer3 = MatMul(hidden_size, output_size)

    def forward(self, x):
        x = self.ReLU(self.layer1(x)) # [B, L, H]
        padded = pad(x, (0, 0, self.atten_size, self.atten_size, 0, 0))
        x_nei = [torch.roll(padded, k, dims=1)
                  for k in range(-self.atten_size, self.atten_size + 1)]
        x_nei = torch.stack(x_nei, dim=2)
        x_nei = x_nei[:, self.atten_size:-self.atten_size, :, :]
        x_nei = x_nei + self.pos_encoding # local positional encoding
        query = self.W_q(x).unsqueeze(2) # [B, L, 1, H]
        keys = self.W_k(x_nei) # [B, L, win, H]
        vals = self.W_v(x_nei)
        atten_scores = torch.sum(query * keys, dim=-1) / self.sqrt_hidden_size
        atten_weights = self.softmax(atten_scores)
        x = torch.sum(atten_weights.unsqueeze(-1) * vals, dim=2) # [B, L, H]
        x = self.ReLU(self.layer2(x))
        x = self.layer3(x) # [B, L, 2] sub-scores
        return x, atten_weights
```

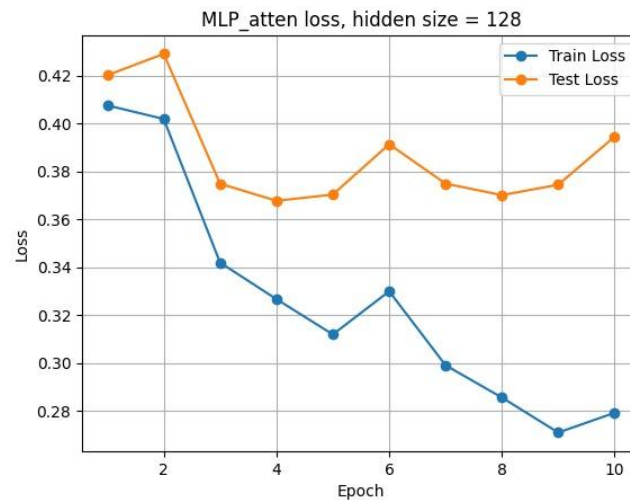
השורות שבהן מחושבים ציוני ה- $\text{attention}(\text{query} \cdot \text{keys})$:

```
query = self.W_q(x).unsqueeze(2) # [B, L, 1, H]
keys = self.W_k(x_nei) # [B, L, win, H]
atten_scores = torch.sum(query * keys, dim=-1) / self.sqrt_hidden_size
atten_weights = self.softmax(atten_scores)
```

משימה 4 תוצאות האימון (hidden size = 128)



accuracy - MLP_atten, hidden size = 128



loss - MLP_atten, hidden size = 128

הערכה מלאה לאחר האימון:

Full evaluation after training:
 Train Loss: 0.2792, Train Acc: 0.8798
 Test Loss: 0.3946, Test Acc: 0.8273

ה-(0.827) test accuracy דומה מאוד ל-MLP ללא (0.828) attention. השכבה אינה משפרת את הדיוק הכולל, משום שרוב הביקורות נשלטות ממילא ע"י מילים רגשיות מובהקות - אך כפי שנראה בהמשך, היא משנה מהותית את ציוני המילים הבודדות (contextualization).

4.1 ניתוח שגיאות והשוואה ל-MLP

הרצנו את אותן ארבע הביקורות (TP, TN, FP, FN) דרך רשת ה-attention, והדפסנו את ציוני תת-הניבוי לכל מילה.

ביקורת 1 - TP (true positive)

"This movie was wonderful and exciting with great acting and a beautiful story"

תווית אמת: positive · ניבוי: positive=1.0000, negative=0.0000

neg_score	pos_score	מילה
-14.3195	21.6949	this
-18.6072	28.3746	movie
-8.2310	12.4900	was
-13.7151	20.6487	wonderful
0.0499	-0.0515	and
-12.6300	19.0439	exciting
0.5732	-0.5480	with
-7.1491	10.5889	great
-11.3531	17.1194	acting
-0.9541	1.5058	and
0.0716	0.5498	beautiful
-3.9304	6.1979	story

ביקורת 2 - TN (true negative)

"This movie was boring slow confusing and disappointing with terrible acting"

תווית אמת: negative · ניבוי: negative=1.0000, positive=0.0000

neg_score	pos_score	מילה
21.0476	-34.0396	this
29.0836	-46.3682	movie
23.7634	-38.3297	was
23.0479	-38.0157	boring
21.8283	-35.8782	slow
23.2549	-38.5979	confusing
0.1725	-0.1670	and
26.4766	-41.9704	disappointing
-0.5888	0.2294	with
22.1330	-36.3424	terrible
24.2916	-38.3240	acting

ביקורת 3 - FP (false positive)

"My friends said the movie was wonderful but my experience was the complete opposite"

תווית אמת: negative · ניבוי: positive=0.9935, negative=0.0065

neg_score	pos_score	מילה
2.0874	-3.4787	my
4.3130	-7.1874	friends
-0.2603	0.1061	said

neg_score	pos_score	מילה
0.3704	-0.3593	the
-18.3857	28.0324	movie
-0.5129	0.8436	was
-13.3271	20.0591	wonderful
-0.3829	0.6206	but
-4.0502	5.8800	my
-0.2154	0.4859	experience
-1.0920	2.1756	was
0.3223	-0.3827	the
1.8050	-2.4484	complete
1.4221	-1.8098	opposite

ביקורת 4 - FN (false negative)

"The beginning was slow and boring but later the story became moving beautiful and excellent"

תווית אמת: positive · ניבוי: negative=0.9914, negative=0.0086

neg_score	pos_score	מילה
1.9916	-3.4044	the
10.1426	-16.5095	beginning
15.9811	-26.0837	was
19.3650	-31.2057	slow
-0.6872	0.3536	and
20.3774	-33.7028	boring
-1.5828	1.1625	but
-0.6061	0.3734	later
-0.8646	0.8325	the
-2.4400	3.3737	story
-0.9163	0.7954	became
-1.2442	1.7626	moving
-19.7178	31.1172	beautiful
-0.6992	0.9385	and
-14.7874	23.3677	excellent

הסבר והשוואה ל-MLP

השינוי הבולט ביותר לעומת ה-MLP הוא שמילים נייטרליות ("this", "movie", "was") מקבלות כעת ציון חזק לפי ההקשר המקומי שלהן: ב-TP הן חיוביות מאוד ("this" +22, "movie" +28), וב-TN הן שליליות מאוד ("movie" -46, "this" -34). ב-MLP אותן מילים קיבלו ציון קרוב לאפס. זהו ה-contextualization שה-attention מספק - כל מילה סופגת את המטען הרגשי של חלון 11 המילים סביבה.

עם זאת, ה-FP וה-FN עדיין נכשלים. הסיבה: ה-attention כאן מקומי (חלון של 5 מילים מכל צד). ב-FP ("wonderful but my experience was the complete opposite...") המילה "wonderful" וה-"opposite" השוללת אותה מרוחקות ביותר מ-5 מילים, כך שהחלון אינו מגשר ביניהן ו-"wonderful" (+20) "wonderful" נותרת דומיננטית. ב-FN ("...slow and boring but later ... beautiful and excellent") המילים השליליות בפתיחה והחיוביות בסיום רחוקות זו מזו, וההיפוך "but later" אינו מקומי. לכן היפוכי-רגש ארוכי-טווח נותרים מחוץ להישג ידה של שכבת ה-attention המקומית.

4.2 ביקורת תלוית-הקשר - הדגמת יתרון ה-attention

כדי להדגים מקרה שבו ה-attention מצליח בעוד רשת משימה 2 (MLP) נכשלת, בנינו ביקורת קצרה שבה מילה משנה את משמעותה בגלל **שכנתה הצמודה**: "The movie was not good". כאן המילה "not" שוללת את "good", וההיפוך הוא מקומי (מילים צמודות) ולכן נמצא בתוך חלון ה-attention.

רשת ה-MLP: ניבוי positive (שגוי, FP), negative=0.3740, positive=0.6260.

רשת ה-attention: ניבוי negative (TN, נכון), negative=0.6680, positive=0.3320.

השוואת ציוני תת-הניבוי לכל מילה (MLP מול attention):

מילה	MLP_pos	MLP_neg	ATT_pos	ATT_neg
the	0.1079	-0.1067	-0.0066	-0.0681
movie	-0.1127	-0.0720	-0.6389	-0.2229
was	-1.1130	1.0179	-1.0741	0.3613
not	-3.3802	2.8548	-0.0462	-0.0353
good	4.8942	-5.8736	-1.2577	0.4373

ההסבר נעוץ בציון של המילה "good": ב-MLP היא מקבלת ציון חיובי גבוה (+4.89) ללא תלות בהקשר, ולכן גוברת על "not" (-3.38) והניבוי חיובי - שגוי. ב-attention, לעומת זאת, "good" מקבלת ציון שלילי (-1.26 pos מול +0.44 neg): השכנה הצמודה "not" (במרחק מילה אחת, בתוך החלון) הופכת את משמעותה. שימו לב גם ש-"not" עצמה כמעט מתאפסת ב-attention (-0.05), משום שאפקט השלילה "הועבר" אל המילה שאותה היא שוללת. זוהי בדיוק היכולת שחסרה ל-MLP נטול-ההקשר, וכאן ה-attention המקומי מצליח היכן שה-MLP נכשל.

חלק ב' - החלק התיאורטי

שאלה 1 - בחירת ארכיטקטורות רשת

סעיף א: זיהוי דיבור (מאודיו לטקסט). סוג הרשת המתאים הוא RNN או מודל Transformer. מבנה הבעיה הוא Many-to-Many מכיוון שקלט האודיו מיוצג כרצף והפלט הוא רצף תווים או מילים, ואורכם משתנה ואינו חופף אחד לאחד.

סעיף ב: מענה על שאלות. סוג הרשת הנדרש הוא מודל מסוג Transformer או רשת מבוססת Seq2Seq. המבנה הוא Many-to-Many מכיוון שהמודל קורא רצף קלט ומייצר משפט פלט באורך משתנה.

סעיף ג: ניתוח סנטימנט. סוג הרשת המתאים ביותר הוא מודל Transformer (כגון BERT) או רשת RNN. מבנה הרשת במקרה זה הוא Many-to-One. קלט הרשת מורכב מסדרת אסימונים (Tokens) המייצגים את המילים בביקורת, והרשת מעבדת את הרצף כדי להפיק בסופו פלט יחיד, שהוא וקטור הסתברויות עבור הסיווג הסופי.

סעיף ד: סיווג תמונות. סוג הרשת המתאים הוא CNN המשמש כמחלץ מאפיינים (Feature Extractor), ולאחריו רשת Fully Connected המרכיבה מודל MLP עבור קלסיפיקציה מהמרחב הלטנטי. רשת ה-CNN משתמשת בפילטרים מקומיים כדי לזהות תבניות היררכיות ולדחוס את התמונה, ולאחר מכן ה-MLP משתמש בייצוג הלטנטי הזה כדי לבצע את הסיווג הסופי לקטגוריות.

סעיף ה: תרגום מילה בודדת. סוג הרשת המתאים הוא רשת Fully Connected. מבנה הבעיה הוא One-to-One. המילה מוזנת כוקטור בודד וממופה לוקטור פלט בודד המייצג את המילה בשפת היעד, ללא תלות בהקשר רצפי כלשהו.

שאלה 2 - מודל המרה מטקסט לתמונה

סעיף א: תיאור הארכיטקטורה. הרשת תורכב ממבנה כללי של Encoder-Decoder. עבור רכיב ה-Text Encoder נבחר במודל Transformer או ברשת RNN (למשל מודל LSTM). המקודד ימפה את משפט הקלט ויפלוט קואורדינטות בתוך מרחב לטנטי (Latent Space) מצומצם המותאם לעולם התמונות המבוקש (כגון פנים). עבור רכיב ה-Image Decoder נבחר בארכיטקטורה של מחולל (Generator) מתוך מודל GAN או מפענח של מודל VAE, המבוססים על שכבות קונבולוציה הפוכות (Transposed Convolutions). מפענח זה ייקח את הייצוג הלטנטי המרוכז ויפרוס אותו בצורה מדורגת עד לקבלת התמונה המלאה ברזולוציה הנדרשת. ארכיטקטורה זו מתאימה למשימה כיוון שהיא מסוגלת לדחוס הבנה סמנטית של טקסט למרחב נמוך-ממד, וממנו לייצר ייצוג ויזואלי עשיר בעל כושר ביטוי גבוה.

סעיף ב: שימוש במנגנון תשומת לב עבור פירוט עדין. כדי לשייך חלקי טקסט ספציפיים לאזורים מוגדרים בתמונה, נשלב במערכת מנגנון Cross-Attention. ארבעת הקודים הלטנטיים המייצגים את ארבעת רבעי התמונה יישמשו בתור Queries במנגנון. במקביל, ה-Text Encoder יפיק ייצוג וקטורי נפרד עבור כל מילה במשפט הקלט, ווקטורים אלו יישמשו בתור Keys ו-Values. באמצעות חישוב מכפלות פנימיות (Dot Product) בין ה-Queries ל-Keys, הרשת תחשב מטריצת משקלי תשומת לב. מנגנון זה מאפשר לכל רבע בתמונה ללמוד באופן עצמאי אילו מילים בטקסט רלוונטיות לעיצוב הפיקסלים שלו, ובכך לתמוך בתיאורים מפורטים ועדינים המקשרים ישירות בין מילים ספציפיות לאזורים מרחביים.

שאלה 3 - רשתות קונבולוציה

סעיף א: גודל מפת הפלט עבור קלט בממדים $128 \times 128 \times 1$.

היגיון חישוב הממדים בקונבולוציה: כאשר אנו מניחים פילטר בגודל k על קלט בגודל L , המיקום הראשון של הפילטר תופס בדיוק k פיקסלים. כתוצאה מכך, נותרים לנו להמשך ההזזה של הפילטר לאורך השורה בדיוק $L-k$ פיקסלים (בתוספת הריפוד $2p$). את יתרת הפיקסלים הזו אנו מחלקים בגודל הצעד s כדי למצוא כמה קפיצות מלאות נוספות ניתן לבצע. לבסוף, אנו מוסיפים 1 שמייצג את המיקום הראשון והבסיסי של הפילטר בתחילת השורה. לכן המונה שואל כמה פיקסלים נותרו לאחר הנחת הפילטר הראשון, ולא כמה פיקסלים נגרעו בסך הכל.

בשכבה הראשונה החישוב הוא:

$$\lfloor (128 - 3 + 0) / 2 \rfloor + 1 = 62 + 1 = 63$$

בשכבה השנייה החישוב הוא:

$$\lfloor (63 - 5 + 4) / 1 \rfloor + 1 = 62 + 1 = 63$$

בשכבה השלישית החישוב הוא:

$$\lfloor (63 - 3 + 2) / 1 \rfloor + 1 = 62 + 1 = 63$$

בשכבה הרביעית החישוב הוא:

$$\lfloor (63 - 5 + 0) / 2 \rfloor + 1 = 29 + 1 = 30$$

הסבר לגבי החישוב בשכבה הרביעית: גודל הקלט הנכנס לשכבה זו הוא 63. גודל הפילטר הוא 5 והריפוד הוא 0. לפי ההיגיון שהוצג, לאחר הנחת הפילטר הראשוני תופסים 5 פיקסלים, ונותרים בדיוק 58 פיקסלים פנויים להמשך ההחלקה (63 פחות 5). כאשר מחלקים את 58 הפיקסלים הנותרים בצעד שגודלו 2, מקבלים בדיוק 29 קפיצות מלאות. כאשר מוסיפים את המיקום הראשון, התוצאה הסופית היא 30. לכן פלט הרשת הוא 30x30.

סעיף ב: שדה הראייה של נירון מרכזי בשכבה הסופית.

בשכבה הראשונה החישוב הוא (מתחילים משדה ראייה בסיסי של 1):

$$1 + (3 - 1) \times 1 = 3$$

בשכבה השנייה החישוב הוא (מכפלת הצעדים של השכבות הקודמות היא 2):

$$3 + (5 - 1) \times 2 = 11$$

בשכבה השלישית החישוב הוא (מכפלת הצעדים של השכבות הקודמות נותרת 2):

$$11 + (3 - 1) \times 2 = 15$$

בשכבה הרביעית החישוב הוא (מכפלת הצעדים של השכבות הקודמות נותרת 2):

$$15 + (5 - 1) \times 2 = 23$$

גודל שדה הראייה הסופי של כל נירון בשכבה האחרונה הוא 23x23.

שאלה 4 - רשתות מבוססות תשומת לב

חוסר תלות בסדר האסימונים (Permutation Invariant). הפעולות בשכבת Self-Attention מבוססות על חישוב משקלים באמצעות מכפלה פנימית (Dot Product) בין ה-Queries ל-Keys של כל האסימונים (Tokens). פעולות אלו ממוצעות ומסכמות את ה-Values בצורה משוקללת ללא תלות מספרית במיקום הפיזי שלהם בתוך המטריצה, מאחר שמדובר באלגברה של קבוצה ולא של סדרה. במקביל, שכבות ה-Feed-Forward מופעלות על כל אסימון קלט בנפרד ובאופן עצמאי לחלוטין באמצעות רשת Fully Connected קבועה. מסיבה זו, אם נשנה את סדר אסימוני הקלט, הרשת תבצע את אותם החישובים בדיוק עבור כל אסימון, והתוצאה בפלט תהיה זהה לחלוטין בערכיה, למעט העובדה שהיא תופיע בסדר המעורבב החדש בהתאמה. תכונה זו מגדירה פונקציה שהיא Permutation Invariant.

שבירת התכונה בעזרת Positional Encoding. הוספה של רכיב קידוד מיקום (Positional Encoding) משנה את וקטורי הקלט המקוריים על ידי חיבור וקטורים המכילים תבניות מחזוריות ייחודיות המשתנות כפונקציה ישירה של האינדקס המדויק ברצף. לאחר שילוב קידוד זה, אם נשנה את סדר אסימוני הקלט, כל אסימון יתחבר לוקטור מיקום חדש ושונה לחלוטין התואם למיקומו הנוכחי. שינוי מספרי זה בוקטורי הקלט ישפיע בצורה ישירה על תוצאות המכפלות הפנימיות, על משקלי התשומת לב (Attention Weights) ועל השכבות הבאות. כתוצאה מכך, הערכים המספריים שיופקו בפלט ישתנו לחלוטין, והתכונה של אי-תלות בסדר הקלט בתוך מודל ה-Transformer תתבטל לחלוטין.